

```

/*package java.de.hsbi.lockgame.logic;

import de.hsbi.lockgame.logic.GameState;
import de.hsbi.lockgame.model.*;
import org.junit.jupiter.api.Test;
import java.util.List;
import java.util.ArrayList;

import static org.junit.jupiter.api.Assertions.*;

public class GameStateTest {

    private Level createEmptyLevel(int width, int height, Position
snakeStart, List<Pin> pins) {
        CellType[][] cells = new CellType[width][height];
        for (int x = 0; x < width; x++) {
            for (int y = 0; y < height; y++) {
                cells[x][y] = CellType.EMPTY;
            }
        }
        return new Level(width, height, cells, pins, snakeStart);
    }

    private Snake createSnakeAt(Position pos) {
        List<Position> body = new ArrayList<>();
        body.add(pos);
        return new Snake(body);
    }

    @Test
    public void testInitialStateRunning() {
        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of());
        Snake snake = createSnakeAt(new Position(2, 2));
        GameState state = new GameState(level, snake, List.of(),
GameState.Status.RUNNING, Direction.NONE);

        assertEquals(GameState.Status.RUNNING, state.status());
        assertEquals(Direction.NONE, state.pendingDirection());
    }

    @Test
    public void testNoMovementWhenDirectionNone() {
        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of());
        Snake snake = createSnakeAt(new Position(2, 2));
        GameState state = new GameState(level, snake, List.of(),
GameState.Status.RUNNING, Direction.NONE);

```

```

        GameState next = state.tick();
        assertEquals(snake.head(), next.snake().head());
    }

    @Test
    public void testMoveRight() {
        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of());
        Snake snake = createSnakeAt(new Position(2, 2));
        GameState state = new GameState(level, snake, List.of(),
GameState.Status.RUNNING, Direction.RIGHT);

        GameState next = state.tick();
        assertEquals(new Position(3, 2), next.snake().head());
    }

    @Test
    public void testOutOfBoundsLoss() {
        Level level = createEmptyLevel(3, 3, new Position(2, 1),
List.of());
        Snake snake = createSnakeAt(new Position(2, 1));
        GameState state = new GameState(level, snake, List.of(),
GameState.Status.RUNNING, Direction.RIGHT);

        GameState next = state.tick();
        assertEquals(GameState.Status.LOST_OUT_OF_BOUNDS,
next.status());
    }

    @Test
    public void testWallBlocksMovement() {
        CellType[][] cells = new CellType[3][3];
        for (int x = 0; x < 3; x++) {
            for (int y = 0; y < 3; y++) {
                cells[x][y] = CellType.EMPTY;
            }
        }
        cells[1][1] = CellType.WALL;

        Level level = new Level(3, 3, cells, List.of(), new
Position(0, 1));
        Snake snake = createSnakeAt(new Position(0, 1));
        GameState state = new GameState(level, snake, List.of(),
GameState.Status.RUNNING, Direction.RIGHT);

        GameState next = state.tick();
        assertEquals(Direction.NONE, next.pendingDirection());
    }

```

```

        assertEquals(new Position(0, 1), next.snake().head());
    }

    @Test
    public void testSelfCollisionLoss() {
        List<Position> body = List.of(
            new Position(2, 2),
            new Position(2, 3),
            new Position(3, 3),
            new Position(3, 2)
        );
        Snake snake = new Snake(body);

        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of());
        GameState state = new GameState(level, snake, List.of(),
GameState.Status.RUNNING, Direction.DOWN);

        GameState next = state.tick();
        assertEquals(GameState.Status.LOST_SELF_COLLISION,
next.status());
    }

    @Test
    public void testPinBlocksWrongDirection() {
        Pin pin = new Pin(new Position(3, 2), Pin.State.LOW,
Direction.UP);
        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of(pin));
        Snake snake = createSnakeAt(new Position(2, 2));

        GameState state = new GameState(level, snake, List.of(pin),
GameState.Status.RUNNING, Direction.RIGHT);
        GameState next = state.tick();

        assertEquals(Direction.NONE, next.pendingDirection());
        assertEquals(new Position(2, 2), next.snake().head());
    }

    @Test
    public void testPinActivatesCorrectDirection() {
        Pin pin = new Pin(new Position(3, 2), Pin.State.LOW,
Direction.RIGHT);
        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of(pin));
        Snake snake = createSnakeAt(new Position(2, 2));

```

```

        GameState state = new GameState(level, snake, List.of(pin),
GameState.Status.RUNNING, Direction.RIGHT);
        GameState next = state.tick();

        assertTrue(next.pins().get(0).state().isSet());
        assertEquals(new Position(2, 2), next.snake().head());
    }

    @Test
    public void testWinConditionAllPinsHigh() {
        Pin pin = new Pin(new Position(3, 2), Pin.State.LOW,
Direction.RIGHT);
        Level level = createEmptyLevel(5, 5, new Position(2, 2),
List.of(pin));
        Snake snake = createSnakeAt(new Position(2, 2));

        GameState state = new GameState(level, snake, List.of(pin),
GameState.Status.RUNNING, Direction.RIGHT);
        GameState next = state.tick();

        assertEquals(GameState.Status.WON, next.status());
    }
}
*/

```